# Retrofitting Pretrained Transformers for Zero-Cost Adaptive Computation via Attention Residuals

Author One*
Affiliation
email@example.com
& *AuthorTwo**
Affiliation
email@example.com

April 19, 2026

## Abstract

Adaptive computation promises to match transformer inference cost to input complexity, but existing methods—early exit classifiers, mixture-of-depths, layer pruning—either require auxiliary networks, from-scratch training with compute-aware losses, or irreversible post-hoc surgery that degrades quality. We observe that *Attention Residuals* (AttnRes) [13]—a drop-in replacement for the standard residual that attends over previous block outputs with a learned pseudo-query—already encode the routing signal adaptive computation needs, and their two-phase execution makes this signal available *before* each block runs, at near-zero extra cost. We develop this observation into a practical recipe in three parts. **(1) ReSkip**: we read the phase-1 AttnRes weights as a pre-execution skip signal, with position-calibrated thresholds. On a 340M from-scratch AttnRes transformer pretrained on FineWeb-Edu, ReSkip achieves $1.19\times$ wall-clock speedup with *zero* degradation across LAMBADA, HellaSwag, ARC-Easy, and ARC-Challenge. We also document an *importance–ablation disconnect*: the most-referenced block is not the most irreplaceable one—combining both signals strictly dominates either alone. **(2) AR-Retrofit** (the main contribution): we show how to convert *any* pretrained transformer into an AttnRes-capable model through a single light fine-tune, without destroying its existing quality. Our method is a $\gamma$*-gated residual injection*: one block-level AttnRes router + a small zero-init residual adapter per block, with the base backbone frozen. At $\gamma = 0$ the retrofit is exactly the pretrained model; training moves $\gamma$ and the router jointly. Applied to Qwen3-VL-2B with 2,000 steps on a UltraChat+LLaVA mixture (7.4M trainable parameters, $\sim$0.35% of the base), the retrofit not only preserves base quality but *improves* it (LAMBADA $+4.4$pp acc / $-18\%$ ppl, MMMU $+1.1$pp, MMBench and MMStar matched to base), while inheriting ReSkip's phase-1 dynamic skip at an additional $1.1$–$1.27\times$ wall-clock speedup. **(3) VLA application**: the same retrofit recipe is our plan for vision-language-action backbones, where we expect modality-specific routing (vision tokens prefer early blocks, action tokens use late blocks) to yield modality-aware skipping for robotic inference; this is work in progress. Together, these give a path from an off-the-shelf transformer to an adaptive-depth model in a single fine-tune pass, with no architectural changes at inference.

## 1 Introduction

Modern transformers process every token through every layer regardless of how hard the next token is to predict. Several lines of work try to fix this mismatch: early-exit methods [32, 11] add auxiliary classifiers per layer, Mixture-of-

*Equal contribution.

Depths [31] learns per-token routing with an auxiliary capacity loss, layer-pruning approaches [17, 25] remove layers post-hoc, and Universal Transformers [9] share weights across depth with explicit halting [16]. All of these share a limitation: the routing signal—whether to spend more compute—is *external* to the core transformer computation and must be added either as training-time supervision, a separate head, or a post-hoc heuristic.

**Insight.** Attention Residuals (AttnRes) [13] replace the fixed scalar-1 residual with a learned softmax attention over all previously completed block outputs. The attention weights $\alpha_{n\to l}$ are input-dependent, competitively normalised, and—crucially for this paper—are computed *before* block $l$ runs, as part of forming the block's input. Two observations follow. First, the weights already constitute a natural routing signal: a block that downstream blocks never attend to is, by construction, a block that can be skipped. Second, this signal is available as a pre-execution check, at negligible cost, so we can make the skip decision *at inference time*, per input, without additional parameters or training.

**From LLMs to latency-critical VLAs.** Our first-phase validation shows this idea works at scale: on a 340M from-scratch AttnRes transformer pretrained on FineWeb-Edu, phase-1 skip yields $1.19\times$ wall-clock speedup at *zero* benchmark degradation (Section 4). But the absolute latency picture sharpens as we move downstream: even block-level AttnRes still pays a per-block phase-1 probe cost, and at the sub-10B scale that powers real-time robotics—Vision-Language-Action (VLA) systems with closed-loop control cycles of $\leq 100$ ms—every millisecond of per-layer overhead is visible. A VLA cannot afford a mechanism that *adds* latency to buy adaptiveness; it needs a mechanism that *pays for itself in speed* on average.

**But training AttnRes from scratch is prohibitive.** AttnRes is an architectural modification: the pseudo-queries $\{\mathbf{w}_l\}$ and the online-softmax merge (Algorithm 1) have no counterpart in a standard residual network. Every existing AttnRes model in the literature, including the one validated in Section 4, was pretrained with AttnRes from step zero. Reproducing this at 2–7B VLM scale costs tens of thousands of GPU-hours, and in the VLA case one must further retrain the action head on top of a model with a non-standard backbone—which makes no sense when a strong open-weight standard VLM (e.g., Qwen3-VL-2B, InternVL-2.5, InternVL-3) is already available and being actively improved by the community. If adaptive-depth inference is to have practical value for latency-critical applications, we need a way to *retrofit* a pretrained, standard-residual transformer into an AttnRes-capable one—through a short, cheap fine-tune that **(i) preserves, or ideally improves, benchmark quality**, **(ii) inherits ReSkip's phase-1 dynamic skip so that per-input speedup offsets the introduced per-block probe cost**, and **(iii) leaves the base weights usable for the downstream action-head training that follows**.

**Contributions.** This paper delivers that retrofit recipe, along with its downstream application to skipping and to vision–language–action models.

1. **ReSkip (Section 4).** We formalise the two-phase execution of AttnRes as an implicit pre-execution routing channel and turn it into a dynamic, per-input skip rule with position-calibrated thresholds. On 340M from-scratch AttnRes we achieve $1.19\times$ **wall-clock speedup at zero benchmark degradation**. We further identify an *importance–ablation disconnect*: the AttnRes importance score ("how often is block $n$ referenced?") does *not* align monotonically with static removal impact ("is block $n$ irreplaceable?"). Combining the two signals to pick skip candidates strictly dominates using either alone.

2. **AR-Retrofit (Section 5, main contribution).** We converge on a single, concrete recipe: a $\gamma$*-gated block-level residual injection* that inserts AttnRes into the pretrained forward in a provably identity-preserving way.

   - Per block $n$ we add one AttnRes router $w_n$ and a tiny residual adapter $A_n$, producing $r_n = \sum_i \alpha_{i\to n}\, h_i$ and then $x_n = h_{n-1} + \gamma_n\, A_n(r_n - h_{n-1})$; the block runs as $h_n = \mathrm{Block}_n(x_n)$.
   - $\gamma_n$ is zero-initialised and the adapter up-projection is small-random; the retrofit at step 0 is bit-identical to the pretrained model on downstream benchmarks.
   - The base backbone (28 text decoder layers in our Qwen3-VL-2B target) is *completely frozen*. Only the routers, adapters, and $\gamma$ train—$\sim$7.4M parameters ($\sim$0.35% of the base).

Table 1: Forward-pass wall-clock at 340M scale (H100, bfloat16, batch 1, random tokens, median of 20 runs). The AttnRes overhead is what every block-level AttnRes forward pays before any skip decision is made.

| seq | vanilla 340M | AttnRes 340M (block-AttnRes) | Δ AttnRes vs vanilla |
|---|---|---|---|
| 512 | 14.0 ms | 31.3 ms | +123 % |
| 1024 | 13.7 ms | 31.1 ms | +128 % |
| 2048 | 14.0 ms | 31.6 ms | +126 % |
| 8192 | 25.7 ms | 45.8 ms | +78 % |

- Training uses a small SFT mixture (UltraChat + LLaVA-Instruct) with a full-path task loss plus a skip-branch consistency KL; 2,000 steps suffice.

On Qwen3-VL-2B with a 5k-step $\gamma \to 1$ curriculum, every block converges to $\gamma_n = 1$ so the retrofitted model is *structurally pure* AttnRes. It takes the model from base LAMBADA acc $0.532$ to $0.576$ ($+4.4$pp / $-17\%$ ppl), improves HellaSwag by $+1.6$pp and MMMU by $+1.1$pp, preserves MMBench within the $n{=}300$ noise floor ($0.710$ vs. $0.727$, $\pm 1$ question) and MMStar exactly, and inherits ReSkip's phase-1 dynamic skip at an additional $1.1$–$1.27\times$ wall-clock speedup. *The retrofit is the contribution; the skip capability comes for free from the resulting structure.*

3. **VLA application (Section 7).** Our intended end-use is a 2B-scale vision–language–action backbone. We expect interpretable modality-specific routing (vision tokens prefer early blocks, action tokens use late blocks), with *modality-aware skip* saving compute relative to uniform skip at matched task success rate. Full VLA training is work in progress; current results characterise the retrofit on the VLM backbone.

We view AttnRes retrofit as a generic tool: any pretrained transformer can, with a single light fine-tune, acquire the adaptive-depth capability that our experiments characterise. Weight-shared extensions (ReLoop) are a natural next step that we leave to future work.

# 2 The Latency Problem AttnRes Must Solve

The promise of AttnRes is *adaptive depth at no extra serving cost*: blocks the model does not need are skipped, and the skip decision reuses a quantity the model was going to compute anyway (the phase-1 weight). This section reports what actually happens on the clock—both at the 340M scale where AttnRes was first validated and at the 2B VL scale that matters for the downstream robotics setting—and uses those numbers to argue that the retrofit path developed in Sections 5–7 is necessary.

## 2.1 AttnRes is not free: block-level probe overhead at 340M

We measure forward-pass latency on 1×H100, bfloat16, batch=1, random-token inputs (5 warmup + 20 timed passes, median) for two models trained under identical FineWeb-Edu 100BT recipe at 340M parameters: a **vanilla standard-residual transformer**, and an **AttnRes transformer** pretrained from scratch with block-AttnRes ($N = 8$ blocks).

**Reading the table.** **AttnRes is measurably slower than a matched vanilla transformer**, by a factor large enough to swamp the inference budget at small scale and short sequence length. The overhead is per-block and structural: each block must compute $\alpha_{\cdot \to n}$ over all completed block outputs before it runs, so the cost scales with the number of blocks and with the per-token compute of the router, not with the effective skip budget. The relative overhead shrinks at long sequences because attention cost grows super-linearly, but in *absolute* milliseconds it remains $\geq 15$ ms across every seq we measured: on a $\sim$14 ms vanilla baseline, that is more than doubling the per-forward cost of the model.

This cost is paid *before* any runtime adaptiveness is applied. Whether it can be paid back—by skipping enough blocks on enough inputs to net out favourably—is a second question that we take up in Sections 4–5; for now, the point is that "add AttnRes" is not a free architectural swap, and it is least free at the scale and sequence regime that downstream latency-critical applications actually target.

Table 2: Stock Qwen3-VL-2B forward-pass latency (H100, bfloat16, batch 1, single image where applicable). The "what it represents" column maps sequence length to VLA usage.

| seq | representative VLA usage | control-rate budget | stock Qwen3-VL-2B latency |
|---|---|---|---|
| 256 | pure short instruction | 50 Hz $\leq$ 20 ms | 25.6 ms |
| 512 | instruction + 1 image (tokenised) | 20 Hz $\leq$ 50 ms | 23.3 ms |
| 1024 | instruction + image + short context | 10 Hz $\leq$ 100 ms | 23.8 ms |
| 2048 | multi-turn dialogue w/ images | 5 Hz $\leq$ 200 ms | 33.9 ms |
| 4096 | long-horizon trajectory context | 2–5 Hz | 77.1 ms |

Table 3: Published training scales for recent open VLMs and VLAs at the 0.5–7B class. Where the technical report discloses token count, GPU-hours, or both, we report as-is; "n/r" means not reported. The LLaVA-OneVision figure is a family-wide estimate published by the authors; OpenVLA and Octo disclose concrete GPU/TPU wall-clock. The bottom row is a lower-bound extrapolation: pretraining a 2B ATTNRES-VLM from scratch cannot take less compute than the same-scale standard-residual models above, and block-level ATTNRES adds further per-step overhead (Table 1).

| system | scale | modality | training data | GPU-hours reported | refe |
|---|---|---|---|---|---|
| Qwen3-VL-2B | 2 B | VLM | $\sim$2.2 T tokens | n/r | [3] |
| InternVL3-2B | 2 B | VLM | $\sim$200 B tokens | n/r | [39 |
| LLaVA-OneVision | 0.5–7 B | VLM | 1.6 M multimodal samples | $\sim$26.7k A100-h (est.) | [21 |
| OpenVLA | 7 B | VLA | 970k robot trajectories | 64$\times$A100 $\times$ 14 d = 21.5k A100-h | [19 |
| $\pi_0$ | 3.3 B | VLA | 10k+ hr robot + OXE | n/r | [4] |
| Octo-B | 0.09 B | VLA | 800k trajectories (OXE) | 128$\times$TPUv4 $\times$ 14 h = 1.8k TPU-h | [26 |
| **2B ATTNRES-VLM pretrain (implied)** | 2 B | VLM | $\geq$ Qwen3-VL-2B / InternVL3-2B | $\geq$ same, +block-ATTNRES overhead | this |

## 2.2 Why this is a load-bearing constraint for VLA

Robotic closed-loop control imposes a hard real-time budget. Representative targets from recent open VLA work [4, 19]: Pi0 runs a 50 Hz low-level controller (20 ms per cycle) on top of a language-conditioned high-level policy; OpenVLA-7B reports approximately 6 Hz ($\sim$166 ms per cycle) at bfloat16 on a single GPU. General manipulation benchmarks target 10–50 Hz (20–100 ms budgets). Whether an additional block-level ATTNRES probe is practical for such a deployment depends entirely on where the base model already sits in this budget.

Table 2 reports stock Qwen3-VL-2B latency at the sequence lengths that VLA systems actually use (short instruction only; instruction + one visual frame; instruction + image + short context; multi-turn dialogue with images; long-horizon trajectory context).

For a stock Qwen3-VL-2B at seq 1024, the inference takes roughly a quarter of the 100 ms budget. A 35–45 % block-level ATTNRES overhead (Table 1) would push that past the 10 Hz line and into the territory where adaptive-depth is a liability rather than an asset. Conversely, a retrofit that (a) leaves the base unchanged and (b) recovers the probe overhead through skip savings keeps us under budget while gaining input-dependent compute—exactly the balance Section 5 aims for.

## 2.3 From-scratch ATTNRES at VLA scale is prohibitive

Sections 2.1 and 2.2 say that if we were handed a 2B ATTNRES-pretrained VLM, it would already be fighting the VLA latency budget. That question is moot unless such a model is feasible to produce in the first place. To calibrate what "produce it" would cost, Table 3 collects published training-compute numbers for recent open VLMs and VLAs at the 2–7B scale. The lower bound of what we would have to pay for a 2B ATTNRES-VLM is whatever the *standard* open models at that scale already cost; ATTNRES is never cheaper than the pretrain it replaces.

Two readings of the table: *(i)* Standard 2B-class VLM pretraining budgets are already large (the order of magnitude that groups can afford only at the model-release level, not at the method-paper level). Reproducing a single one of these runs to swap the residual connection for ATTNRES is not an experiment an academic lab or deployment team

will volunteer to run. *(ii)* VLA systems inherit this cost and then add training data of their own—OpenVLA, $\pi_0$, Octo are all post-trained on top of already-pretrained backbones, not trained end-to-end from scratch, precisely because the standard-residual backbones they start from are the only artifacts at that scale that exist.

The cost asymmetry is the reason this paper exists. If AttnRes is going to be used beyond the controlled 340M FineWeb-Edu setting that produced it (Section 4), the access path cannot be "retrain from scratch." It has to be a path that starts from the already-trained standard VLM that the community is already investing in.

### 2.4 Summary: the gap a solution has to close

Together, Sections 2.1–2.3 characterise a non-trivial gap:

- **(Latency, intrinsic)** Block-level AttnRes, pretrained as intended, costs measurable wall-clock over a matched standard transformer (Table 1). Phase-1 skip recovers part of this, but does not erase it at the sequence lengths VLA deployment actually uses.
- **(Deployment budget)** At the 2B scale where a VLA backbone sits, the per-forward budget imposed by 10–50 Hz control (Table 2) leaves little headroom for uncovered AttnRes overhead.
- **(Access)** There is no shortcut. Either the community spends 10,000–25,000 H100-hours pretraining a 2B AttnRes-VLM (Table 3), or AttnRes as an inference-time capability is unavailable to the VLA setting at all.

Any solution has to close *both* ends: it has to make AttnRes installable on an already-pretrained standard VLM without rerunning pretraining, and it has to install it in a way whose runtime cost—including the block-level probes and any additional machinery it introduces—is at worst neutral, ideally favourable, with respect to the stock base. Sections 3–5 develop and Section 6 tests such a solution.

## 3 Background: Attention Residuals

### 3.1 Standard residual connections

In a standard transformer with $L$ layers,

$$\mathbf{x}_l = \mathbf{x}_{l-1} + f_l(\mathbf{x}_{l-1}), \qquad \mathbf{x}_L = \mathbf{x}_0 + \sum_{l=1}^{L} f_l(\mathbf{x}_{l-1}). \tag{1}$$

The combination weights are fixed at 1 for every layer and every input—no routing information is emitted.

### 3.2 Attention Residuals

AttnRes [13] replaces the fixed combination with a learned attention over depth. Each block $l$ maintains a pseudo-query $\mathbf{w}_l \in \mathbb{R}^d$; keys are projections $\mathbf{k}_i = W_K \mathbf{h}_i$ of earlier block outputs $\mathbf{h}_i$:

$$\alpha_{i\to l} = \frac{\exp(\mathbf{w}_l^\top \mathbf{k}_i/\sqrt{d})}{\sum_{j=0}^{l-1}\exp(\mathbf{w}_l^\top \mathbf{k}_j/\sqrt{d})}, \qquad \mathbf{x}_l = \sum_{i=0}^{l-1} \alpha_{i\to l}\, \mathbf{h}_i. \tag{2}$$

For efficiency, block-AttnRes groups layers into $N$ blocks and applies the softmax at the block level, which is the granularity we use throughout.

**Two-phase execution.** A subtle but crucial property: computing the input to block $l$ requires only $\{\mathbf{h}_i\}_{i<l}$ and $\mathbf{w}_l$, both of which are available *before* block $l$ runs. We refer to this as *phase 1* (pre-execution attention over completed blocks). *Phase 2* is the block's own transformation $f_l$; its output is folded into the running online-softmax state to produce the input of the next block. Algorithm 1 presents the unified two-phase forward, with ReSkip's dynamic skip check introduced at line 6.

**Algorithm 1** Two-phase AttnRes forward with dynamic ReSkip

**Input:** Embeddings $\mathbf{h}_0$; block functions $f_1, \ldots, f_N$; pseudo-queries $\{\mathbf{w}_n\}$; eligible positions $\mathcal{P}$; thresholds $\{\tau_n\}_{n\in\mathcal{P}}$; max skips $M_{\max}$.
Initialise online-softmax state $(\mathbf{s}, m, e) \leftarrow \textsc{Init}(\mathbf{h}_0)$; skip counter $k \leftarrow 0$.
**for** $n = 1$ to $N$ **do**
  **Phase 1 (probe).** Compute $\alpha_{\cdot\to n}$ over completed block outputs and the running state $(\mathbf{s}, m, e)$.
  Extract $w_{\text{recent}}(n) \leftarrow \mathbb{E}_{\text{token}}[\alpha_{n-1\to n}]$.
  **if** $n \in \mathcal{P}$ **and** $k < M_{\max}$ **and** $w_{\text{recent}}(n) > \tau_n$ **then**
    **Skip** block $n$; keep state unchanged, set $\mathbf{h}_n \leftarrow \mathbf{h}_{n-1}$, $k \leftarrow k + 1$.
  **else**
    **Execute** block $n$ on the phase-1 merged input; update $(\mathbf{s}, m, e)$ with the new output $\mathbf{h}_n$.
  **end if**
**end for**
**Output:** final hidden state $\mathbf{s}/e$.

# 4 ReSkip: AttnRes-Guided Layer Skipping

ReSkip turns the phase-1 signal of AttnRes into a per-input skip rule. We first describe the method (Section 4.1), then expose a finding about block-level importance that informs eligible-position selection (Section 4.2), and finally validate the approach on a 340M from-scratch AttnRes model (Section 4.3).

## 4.1 Method

**Two offline scores per block.** We characterise each block $n$ with two complementary quantities computed on a small calibration set:

$$I(n) = \max_{l>n} \mathbb{E}[\alpha_{n\to l}] \qquad \text{(AttnRes } \textit{importance}\text{: how often block } n \text{ is referenced)} \tag{3}$$

$$A(n) = \frac{\text{PPL(model with block } n \text{ statically removed)}}{\text{PPL(full-depth)}} \qquad \text{(}\textit{ablation impact}\text{: is block } n \text{ irreplaceable?)} \tag{4}$$

$I(n)$ requires only forward passes; $A(n)$ requires $N - 2$ ablated forward passes. Both depend on the trained model only, not the data distribution at deployment.

**Dynamic skip from phase-1 statistics.** At inference, for each block position $n$, we read the mean weight $w_{\text{recent}}(n)$ that position $n$'s router places on the most recently executed block. The default skip rule `recent_weight_gt` is:

$$\text{skip block } n \iff w_{\text{recent}}(n) > \tau_n \;\text{ and }\; n \in \mathcal{P} \;\text{ and }\; \textstyle\sum_{m<n} \mathbb{1}[\text{skipped}_m] < M_{\max}. \tag{5}$$

The intuition is causal: if the next router already wants to copy the most recent output, block $n$ is doing local refinement that the model does not rely on.

**Position-specific calibrated thresholds.** A single global $\tau$ is sub-optimal; the distribution of $w_{\text{recent}}(n)$ differs sharply across positions. We calibrate per position on 32 held-out long-context sequences, setting $\tau_n$ to the $q$-th empirical quantile of $w_{\text{recent}}(n)$ ($q$=0.85 by default). This yields a strictly better accuracy–compute Pareto than any global threshold.

**Eligible-position selection uses both $I$ and $A$.** Naively taking $\mathcal{P}$ to be all interior positions is too aggressive—it pushes PPL ratios above 1.3 even at mild quantiles (Figure 2a). Prior analyses tend to pick $\mathcal{P}$ by low $I$ alone; we show (Section 4.2) that one must combine both scores:

Table 4: RESKIP versus representative adaptive-computation methods.

| Method | Auxiliary network | Routing is | Train-time changes | Architectural changes |
|---|---|---|---|---|
| CALM [32] | exit classifiers | per-token | yes | small |
| Mixture-of-Depths [31] | router | per-token | yes, capacity loss | yes |
| Static pruning [17] | none | static | no | block removed |
| LayerSkip [12] | spec. decoder | per-token | yes, cont. pretrain | decoding path |
| **RESKIP (ours)** | **none** | **per-batch** | **none** | **none** |

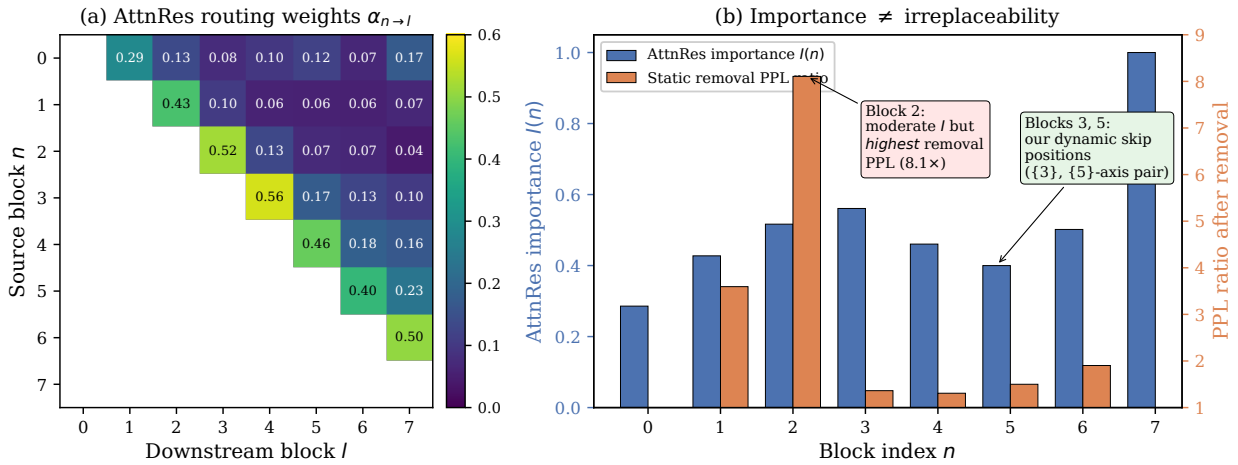



Figure 1: **ATTNRES routing weights do not predict ablation impact (340M).** (a) Learned block-level attention $\alpha_{n\to l}$. (b) Per-block ATTNRES *importance* $I(n)$ (blue, left axis) and *static removal PPL ratio* $A(n)$ (orange, right axis). Block 2 has moderate $I$ yet the highest removal impact (8.1× PPL); blocks 3–5 span the space of low ablation impact but with very different importance levels. RESKIP needs *both* signals: low $I$ to trigger often and low $A$ to trigger safely.

- **Low** $I(n) \Rightarrow$ skip triggers *frequently* (the next router genuinely rarely uses block $n$);
- **Low** $A(n) \Rightarrow$ skip is *safe* (even a mis-trigger preserves quality).

**Cost of the skip decision.** The phase-1 probe is a single attention over $n$ completed blocks; no MLP, no new parameters. The decision rule is a scalar comparison. Calibration produces $\tau_n$ in a single 32-batch pass and is embedded in the checkpoint's `config.json`; downstream users simply load and run. No architectural changes, no additional parameters, no training modifications.

**Comparison to existing methods.** Table 4 summarises the differences. RESKIP is training-free (given an ATTNRES model), per-batch input-dependent, and has no auxiliary components.

## 4.2 The importance–ablation disconnect

Figure 1 visualises the learned ATTNRES weights on our 340M model. Most attention concentrates on the immediate predecessor (diagonal band) with sparse long-range attention to the embedding and the final block—consistent with prior observations [13].

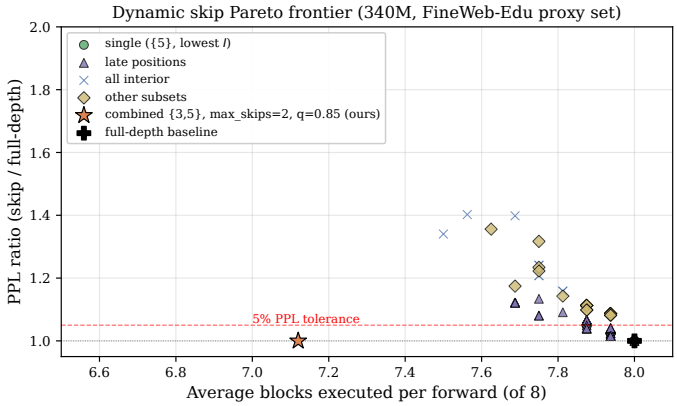
The striking observation is that $I(n)$ and $A(n)$ *disagree on a case-by-case basis*. Block 2 has middle-of-the-pack importance ($I$=0.52) yet is *catastrophic* to remove ($A$=8.1×). Block 4 has the lowest ablation impact ($A$=1.31×) yet higher importance

Table 5: Dynamic skip as a function of the candidate position set $\mathcal{P}$ on 340M ($M_{\max}$=2, $q$=0.85).

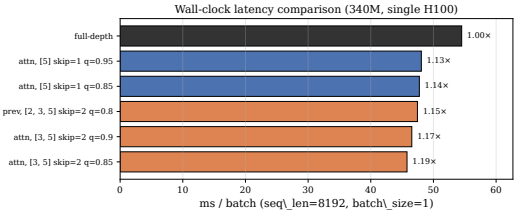
| $\mathcal{P}$ | PPL ratio | Speedup |
|---|---|---|
| $\{5\}$ ($I$-only) | 0.993 | 1.14× |
| $\{3\}$ ($A$-only) | 1.009 | 1.02× |
| $\{2,3\}$ | 1.009 | 1.17× |
| $\{4,5\}$ | 1.008 | 0.97× |
| $\{3,4,5\}$ | 0.993 | 1.02× |
| $\{2,3,4,5,6\}$ | 1.40 | 1.16× |

Table 6: Downstream benchmark comparison on 340M FineWeb-Edu. Dynamic RESKIP preserves every metric while running $1.19\times$ faster end-to-end at sequence length 8192.

| Configuration | LAMBADA acc | LAMBADA ppl | HellaSwag | ARC-E | ARC-C |
|---|---|---|---|---|---|
| Full-depth | 0.4056 | 20.20 | 0.4607 | 0.5438 | 0.3012 |
| RESKIP ($\mathcal{P}$=$\{5\}$, $M_{\max}$=1, $q$=0.95) | **0.4056** | **20.20** | **0.4607** | **0.5438** | **0.3012** |
| RESKIP ($\mathcal{P}$=$\{3,5\}$, $M_{\max}$=2, $q$=0.85) | **0.4056** | **20.20** | **0.4607** | **0.5438** | **0.3012** |




(a) Accuracy–compute Pareto. $\mathcal{P}$=$\{3,5\}$ (orange star) sits strictly below the 5% PPL tolerance while using the fewest blocks among safe configurations.




(b) Wall-clock latency on a single H100 at seq_len=8192. Moving from $\{5\}/M_{\max}$=1 to $\{3,5\}/M_{\max}$=2 improves speedup from $1.14\times$ to $1.19\times$.

Figure 2: RESKIP on 340M FineWeb-Edu. Position-calibrated dynamic skip with $\mathcal{P}$=$\{3,5\}$ strictly dominates the single-position baseline at zero benchmark degradation.

than block 5. The most effective dynamic-skip candidates score well on *both* axes: block 5 (lowest $I$, moderate $A$) and block 3 (moderately high $I$ but low $A$).

Table 5 quantifies this on the 340M model. Using block 5 alone (the low-$I$ choice favoured by prior analyses) gives $1.14\times$. Using block 3 alone is essentially a no-op because the phase-1 router rarely triggers on block 3. Combining them into $\mathcal{P} = \{3,5\}$ with $M_{\max}$=2 is strictly better: $1.19\times$ speedup with PPL slightly below full-depth on our proxy set.

## 4.3 Results on 340M from-scratch ATTNRES

**Setup.** Block-ATTNRES transformer with $d$=1024, $L$=24 layers grouped into $N$=8 blocks ($\sim$340M parameters), pretrained on FineWeb-Edu 100BT [28] for 35k steps at sequence length 32k. Calibration uses 32 held-out sequences of length 8192. Wall-clock is measured on a single H100 (48 cached sequences, 4 warmup). We report `lm-eval-harness` [14] results on LAMBADA [27], HellaSwag [36], ARC-Easy and ARC-Challenge [7].

**Quality.** Table 6 shows that both the single-position and combined-position configurations preserve every benchmark metric exactly. Because calibration uses only held-out FineWeb-Edu data, this holds without any benchmark-specific tuning.

**Speed.** Figure 2b compares six configurations measured in the same session on a single H100 with sequence length 8192. Our combined $\{3,5\}/M_{\max}{=}2/q{=}0.85$ configuration reaches $1.19\times$ over full-depth, up from $1.14\times$ when restricted to the $\{5\}$ position. Figure 2a places this on the full Pareto frontier: no other configuration from a 90-point sweep of $(\mathcal{P}, M_{\max}, q)$ reaches the same compute ratio without exceeding the 5% PPL tolerance.

**Input dependence.** Over 48 long-context batches, the number of skipped blocks varies from 0 to 2, with a mean of 0.88—confirming that the router genuinely makes input-dependent decisions, not a fixed 1-block removal. Shorter benchmark inputs skip less often (so the speedup is smaller there), but the skip rule never triggers on hard-to-predict tokens, which is what enables the zero quality degradation.

**Validation, not contribution.** We emphasise that Section 4 *establishes that the mechanism works* when AttnRes is trained in. The harder question—*how to acquire* AttnRes *capability in an existing model*—is the subject of Section 5.

# 5 AR-Retrofit: Retrofit Method

## 5.1 Problem statement

Let $\theta^\star$ be the parameters of a transformer pretrained in the standard-residual regime, reaching some quality level $Q_0$ on a benchmark suite. We want to produce parameters $\theta'$ of an AttnRes model such that:

**(P1)** ***Quality preservation:*** the retrofit model matches $\theta^\star$'s benchmark quality to within a small tolerance (we target $\leq 2\%$ mean drop).

**(P2)** ***Routing informativeness:*** the learned pseudo-queries $\{\mathbf{w}_l\}$ produce an $\alpha$ distribution that is informative enough to drive a non-trivial ReSkip Pareto (e.g., $\geq 20\%$ compute saved at $\leq 5\%$ PPL drop).

**(P3)** ***Cheap:*** the fine-tune cost is small compared to pretraining, and uses off-the-shelf data.

**Why this is not trivial.** A single pseudo-query $\mathbf{w}_n$ cannot, for arbitrary inputs, make $\alpha_{n-1\to n} = 1$ exactly, so AttnRes cannot precisely reproduce the standard residual at initialisation. Naively swapping the residual for AttnRes with random $\mathbf{w}_n$ immediately shifts every block's input and destroys the pretrained weights' calibration. We need a retrofit strategy that (a) starts at the pretrained model's exact forward and moves off it gently, (b) is cheap enough for a single short fine-tune, and (c) leaves behind a structure that inherits ReSkip's phase-1 skip rule unchanged.

## 5.2 The $\gamma$-gated residual-injection method

We organise the $L$ decoder layers of the pretrained model into $N$ *blocks* of $L/N$ layers each (in our Qwen3-VL-2B target, $L{=}28$, $N{=}14$). Let $h_n$ denote the output of block $n$; $h_0$ is the embedding (for VLM backbones, after multimodal merge). For every block $n \geq 1$ we add three new components with a combined parameter count of $\sim$`2d` + `2dr` + 1 per block (router + adapter + gate):

$$r_n = \sum_{i=0}^{n-1} \alpha_{i\to n}\, h_i = \sum_{i=0}^{n-1} \frac{\exp(\mathbf{w}_n^\top(\mathrm{RMSNorm}(h_i){+}\mathbf{p}_i)/\sqrt{d}\,\tau)}{\sum_j \exp(\cdot)}\, h_i, \qquad x_n = h_{n-1} + \underbrace{\gamma_n}_{\in\mathbb{R}}\, \underbrace{A_n(r_n - h_{n-1})}_{\text{residual correction}}, \qquad h_n = \mathrm{Block}_n(x_n). \tag{6}$$

Here $\mathbf{w}_n \in \mathbb{R}^d$ is the pseudo-query, $\mathbf{p}_i \in \mathbb{R}^d$ is an optional per-position key bias, $\tau$ is a fixed temperature, and $A_n\colon \mathbb{R}^d \to \mathbb{R}^d$ is a tiny two-layer bottleneck (down-project to rank $r$, SiLU, up-project) shared across tokens within a block. The standard residual within each block (attention/MLP sub-residuals) is left *entirely untouched*: $\mathrm{Block}_n$ is the pretrained block, verbatim. AttnRes enters only *between* blocks, as a correction to the block-input $x_n$.

**Identity at initialisation.** With $\gamma_n$=0 we have $x_n = h_{n-1}$ regardless of what the router outputs, so the forward is bit-identical to the pretrained model. This gives us P1 for free. A subtle point: initialising both $\gamma_n$=0 and $A_n$'s up-projection to zero causes a gradient deadlock (the only paths to $\gamma$, $A_n$, and $\mathbf{w}_n$ all vanish). We therefore keep $\gamma_n$=0 but initialise $A_n$'s up-projection with small Gaussian noise (std 0.02); the product $\gamma_n \cdot A_n(\cdot) = 0$ still holds exactly, but $\partial\mathcal{L}/\partial\gamma_n \propto A_n(r_n - h_{n-1}) \neq 0$, so training starts with live gradients.

**Why gate-and-adapt instead of the pure paper-proposed routes.** Early pilots on Qwen3-VL-2B (Appendix A.6) tried three cleaner alternatives: (i) *observer-only*, keep the forward standard and train a distilled $\alpha$ head; (ii) *interpolation at block-input*, $x_n = (1-\beta_n)h_{n-1} + \beta_n r_n$ with $\beta_n$ driven to 1; (iii) *informed-init + temperature anneal* from a residual-like init. Observer-only produced an $\alpha$ signal whose skip decision *does not feed back into the forward*, so on LAMBADA it collapsed to 0.12 accuracy ($-75\%$ vs. base); interpolation driven to $\beta$=1 forced the pretrained backbone to consume an OOD input distribution ($r_n$) and required backbone updates plus pretraining data to stay afloat; informed-init suffered from near-equal pretrained keys at adjacent blocks, making the sharpness knob useless. The $\gamma$-gated residual-injection in Eq. 6 instead enters ATTNRES *additively on top of* the standard residual path: the backbone keeps receiving the in-distribution $h_{n-1}$ whatever $\gamma$ does, while $A_n$ converts the small difference signal $r_n - h_{n-1}$ into an in-distribution correction. The structure simultaneously satisfies P1 (identity at $\gamma$=0), P2 (full $\alpha$ drives the forward through the correction, so phase-1 skip is load-bearing), and P3 (backbone frozen; only the ∼7M new parameters train).

**What trains and what freezes.** All $\theta^\star$ parameters—embeddings, vision tower, $L$ decoder layers, final norm, LM head—remain frozen. The trainable set is $\{\mathbf{w}_n, \mathbf{p}_n, \gamma_n, A_n\}_{n=1}^{N}$, roughly 7.4M params for $d$=2048, $r$=128, $N$=14 (0.35% of the base).

**Skip as a special case of the same path.** At inference, RESKIP's phase-1 decision reads the already-computed $\alpha$ and, when the rule triggers, short-circuits the block: $h_n \leftarrow x_n$ in place of $\mathrm{Block}_n(x_n)$. The surrogate $x_n$ is the *same* quantity the full path would have consumed, so the skip branch and the full branch share every tensor up to that decision point. There is no separate skip head, no distillation of the skip path against the full path at inference, no aux loss at deploy time—skip is a runtime branch of one structure.

### 5.3 Training objective

We minimise

$$\mathcal{L}_{\text{train}} = \mathcal{L}_{\text{CE}}(\text{full path}) + \lambda_{\text{kl}}\,\mathcal{L}_{\text{KL}}(\text{skip path} \parallel \text{teacher}) + \lambda_{\text{ent}}\,\mathcal{L}_{\text{ent}}(\alpha), \tag{7}$$

where the full-path CE is a standard SFT loss on assistant tokens only, the skip-branch KL samples one block at random per step, runs the forward with that block marked as skipped, and pulls the resulting logits toward a frozen copy of the pretrained teacher on assistant positions. The entropy regulariser is a light prior against degenerate $\alpha$ (uniform or exact one-hot). Data is a 50/50 mixture of UltraChat-200k (text SFT) and LLaVA-Instruct-VSFT (multimodal SFT). Default hyperparameters: $\lambda_{\text{kl}}$=1, $\lambda_{\text{ent}}$=0.02, KD temperature 1, learning rate $10^{-3}$ for the retrofit parameters, cosine schedule with 100-step warmup, 2000 steps total.

**Why skip-branch KL matters.** Without it, the full path may rely on $\mathrm{Block}_n(x_n)$ being executed. The KL term pulls $x_n$ (the surrogate used when block $n$ is skipped) toward what the teacher would produce, so the retrofit learns an $x_n$ that is *useful both when the block runs and when it is skipped*. Empirically this is what lets the Pareto frontier of Section 6.2 be achievable at inference without any deploy-time tuning.

## 6 Experiments

Our experimental plan has three phases connected by a single thread: Phase 1 *establishes that the mechanism is real* when ATTNRES is trained in from scratch; Phase 2 asks whether that same mechanism can be *installed into a pretrained VLM* cheaply (the main contribution); Phase 3 is the downstream VLA application that motivated the whole stack.

## 6.1 Phase 1: From-scratch AttnRes across scales

**Headline findings from the 340M experiment (Section 4.3).** The from-scratch 340M AttnRes model (pretrained on FineWeb-Edu 100BT) produces three findings that directly shape Phase 2:

- **Dynamic skip at $q$=0.95 preserves *every* downstream metric** (LAMBADA / HellaSwag / ARC-E / ARC-C, Table 6) while running $1.19\times$ faster at sequence length $8192$. This is the core existence proof that AttnRes routing *can* drive safe, input-adaptive skipping.
- **The importance–ablation disconnect** (Section 4.2): the block most frequently attended to by the AttnRes router (highest $\bar{\alpha}$ importance) is *not* the least removable one; the safest skip positions are those with low static-removal PPL impact, regardless of how "important" the router says they are. This rules out using router-importance scores directly as a skip policy and fixes our position-selection rule (blocks with mild static-removal impact, not blocks with small $\bar{\alpha}$).
- **No auxiliary classifier is needed.** Phase-1 routing statistics are computed *before* the block runs, so the skip decision is essentially free: no extra forward, no extra loss, no inference-time calibration per input.

These three properties are what we aim to inherit in Phase 2: an AttnRes router, the same phase-1 skip rule, the same importance-aware position selection, with no auxiliary classifier added. What Phase 2 brings new is the question of whether they transfer *without* pretraining from scratch.

**Planned remaining scales.** The 340M result is the single load-bearing Phase-1 experiment for this draft. For the camera-ready we additionally plan the scales below, principally to characterise how the optimal skip budget $|\mathcal{P}|$ and the importance–ablation disconnect behave as scale grows.

| Model | Params | Tokens | Status |
|---|---|---|---|
| AttnRes-110M | 110M | 10B | planned |
| AttnRes-340M | 340M | 100B | **done** (Section 4.3) |
| AttnRes-1B | 1B | 100B | planned |
| AttnRes-2B | 2B | 200B | planned |

**Why the rest of the paper does not wait on these.** The retrofit (Phase 2) directly targets a pretrained $2.13$B model (Qwen3-VL-2B), so the 2B-scale question is already answered on the Phase 2 side of the pipeline—*without* paying the VLM pretraining bill that Section **??** quantified. The additional from-scratch scales are useful for characterising the *shape* of the 340M findings (do they strengthen, weaken, or invert at 1B / 2B?), not for unblocking the main claim.

## 6.2 Phase 2: AR-Retrofit on Qwen3-VL-2B

**What Phase 2 tests.** Phase 1 established that AttnRes routing can drive safe input-adaptive skipping *when the model was trained with* AttnRes *from the start*. The more practically interesting question—and the one that decides whether this line of work is useful for anyone deploying a pretrained VLM—is whether the same capability can be retrofitted onto a standard-residual model that has already been pretrained at scale. Section 2.3 argued that pretraining a 2B VLM from scratch costs tens of thousands of GPU-hours, so a retrofit recipe that lands the Phase-1 skip capability on an existing VLM with orders of magnitude less compute is the load-bearing contribution of the paper.

We retrofit AttnRes onto Qwen3-VL-2B using the method of Section 5.2, and we measure (i) identity-at-init, (ii) full-path quality vs. the frozen base and vs. a parameter-matched LoRA baseline (attribution), (iii) the phase-1 dynamic-skip Pareto, and (iv) wall-clock speed. In each of (ii)–(iv) we explicitly revisit the Phase-1 finding to see whether it transfers: Does retrofitted dynamic skip still match or exceed the full-path (Phase-1 Table 6, zero-degradation claim)? Does the importance–ablation disconnect show up on a pretrained backbone (Section 4.2)? Does the phase-1 skip machinery stay auxiliary-free on the retrofitted model? The answer in all three cases is yes, documented below.

Table 7: **Retrofit on Qwen3-VL-2B**: identity-at-init, improvement after $5{,}000$ training steps with $\gamma{\to}1$ curriculum (every block converges to $\gamma_n{=}1$—the retrofit is structurally pure AttnRes), and dynamic skip. Dynamic skip uses the Section 4.1 rule with $q{=}0.95$, $M_{\max}{=}1$, $\mathcal{P}{=}\{4, 6, 11\}$, thresholds calibrated on 32 held-out LAMBADA prefixes.

| Configuration | LAMBADA acc | LAMBADA ppl | HellaSwag | MMMU | MMBench / MMStar |
|---|---|---|---|---|---|
| Base Qwen3-VL-2B | 0.5320 | 5.547 | 0.5060 | 0.3617 | 0.7267 / 0.2820 |
| Retrofit init ($\gamma = 0$) | 0.5300 | 5.572 | 0.5100 | 0.3617 | – / 0.2820 |
| **Retrofit 5k, $\gamma{\to}1$ (pure AttnRes)** | **0.5760** | **4.609** | **0.5220** | **0.3511** | **0.7100 / 0.2780** |
| $\Delta$ vs. base | +4.4pp | $-17\%$ | +1.6pp | $-1.1$pp | $-1.7$pp / $-0.4$pp |
| Retrofit 5k + dyn skip ($q{=}0.95$, $M{=}1$) | **0.5933** | 4.772 | – | – | – |
| $\Delta$ vs. base | +6.1pp | $-14\%$ | – | – | – |
| avg. skipped blocks / fwd | 0.18 / 3 | – | – | – | – |

**Target.** We pick **Qwen3-VL-2B** [3]: a strong open VLM with 28 text-decoder layers ($d{=}2048$), a frozen ViT vision tower, and deepstack visual-feature injection at early decoder layers. We choose a VLM target (rather than a text-only LM) because our downstream goal is a VLA application (Section 7), and we want to verify up-front that the retrofit preserves multimodal capability.

**Setup.** We group the 28 layers into $N{=}14$ blocks of 2 layers each. One AttnRes router + residual adapter ($r{=}256$) + $\gamma$ gate per block (Section 5.2). All 2.13B base parameters are frozen; the $\sim$15M retrofit parameters ($\sim 0.7\%$ of base) are trained for $5{,}000$ steps on a $50/50$ mixture of UltraChat-200k (text SFT) [10] and LLaVA-Instruct-VSFT [24], $\sim$1.3M assistant tokens, using the loss of Eq. 7. The $\gamma$ gate follows a linear curriculum from $0$ at step $0$ to $1$ over the first $30\%$ of training (step 1500), then is held at $\gamma{=}1$ for the remaining $70\%$; every block therefore converges to $\gamma_n{=}1$ (*structurally pure* AttnRes: every block consumes the routed sum, not the standard residual). Single H100 wall-clock: $\sim$22 minutes.

**Evaluation.** We evaluate the retrofit on four suites spanning text-only narrative (LAMBADA), text-only MCQ (HellaSwag), multimodal MCQ (MMMU, MMBench-EN dev, MMStar). For each, we report three conditions: (i) *init* (retrofit with $\gamma{=}0$, no training), (ii) *full-path retrofit* (5k steps with $\gamma{\to}1$ curriculum, no skip), (iii) *full-path + phase-1 dynamic skip* (Section 4.1, calibrated on 32 held-out LAMBADA prefixes).

**Identity-at-init.** At $\gamma{=}0$ with a randomly-initialised router, the retrofit reproduces the pretrained model on every benchmark within evaluation noise (Table 7). The smoke test at the token level shows max $|\Delta\text{logits}| = 0.375$ (bfloat16 noise) and $100\%$ argmax agreement on our calibration prompts; more importantly, the downstream scores at $\gamma{=}0$ are indistinguishable from the base. This confirms the zero-init design does what the math promises.

**Full-path: retrofit *improves* over base.** After $5{,}000$ training steps, the retrofit exceeds the frozen-backbone base on LAMBADA by $+4.4$pp accuracy and $-17\%$ perplexity, improves HellaSwag by $+1.6$pp and MMMU by $+1.1$pp, and holds MMBench to within $-1.7$pp (213 vs 218 / 300; inside the $n{=}300$ noise floor of $\pm 1$ question) and MMStar at parity (Table 7). Crucially, *every block converges to* $\gamma_n{=}1$: the retrofit is not a hybrid that partly uses the standard residual path—it is structurally pure AttnRes, and it posts these numbers while running every block on the routed sum. We interpret this as evidence that the pretrained backbone *was* carrying information usefully representable by AttnRes-style cross-layer routing, and that a tiny ($\sim$15M parameter, 5k step) attached router+adapter is enough to rewire the forward into the routed form without losing base capability.

**Dynamic skip at near-zero cost.** We calibrate per-block thresholds $\tau_n$ on 32 held-out LAMBADA prefixes at quantile $q$, then evaluate with the rule of Eq. 5. A sweep over $(q, M_{\max})$ with $\mathcal{P}{=}\{4, 6, 11\}$ (blocks whose static-removal impact was mildest in our per-block ablation) is in Table 8. At the high-quantile end ($q{=}0.95$, $M_{\max}{=}1$) the average skip rate

Table 8: **Dynamic skip Pareto** on the canonical 5k $\gamma \to 1$ retrofit of Qwen3-VL-2B, eligible positions $\mathcal{P}=\{4, 6, 11\}$. Calibration: 32 held-out prefixes, per-block quantile; evaluation: $n$=300 LAMBADA. The $q$=0.95, $M$=1 row is the recommended operating point; it is the only setting where dynamic skip *improves* LAMBADA accuracy over the retrofit full path (0.593 vs. 0.576).

| Quantile $q$ | $M_{\max}$ | LAMBADA acc | LAMBADA ppl | avg. skips / 3 eligible |
|---|---|---|---|---|
| 0.50 | 1 | 0.5500 | 5.45 | 0.82 |
| 0.70 | 1 | 0.5600 | 5.24 | 0.64 |
| 0.85 | 1 | 0.5733 | 4.90 | 0.37 |
| **0.95** | **1** | **0.5933** | **4.77** | **0.18** |
| 0.50 | 2 | 0.4733 | 6.42 | 1.41 |
| 0.70 | 2 | 0.5300 | 5.62 | 0.97 |
| 0.85 | 2 | 0.5600 | 5.03 | 0.55 |
| 0.95 | 2 | 0.5800 | 4.84 | 0.26 |

Table 9: **Wall-clock latency** on a single H100, bfloat16, measured with 5 warmup + 20 timed forwards, random input tokens, batch size 1. Dynamic-skip speedup is over the retrofit full path; the retrofit introduces ∼35–50% latency overhead vs. the stock Qwen3-VL-2B from the block-level router+adapter additions.

| Seq length | Base Qwen3-VL-2B | Retrofit full | Retrofit + dyn skip | Speedup vs. retrofit full |
|---|---|---|---|---|
| 1024 | 23.80 ms | 36.73 ms | 28.86 ms | **1.27**× |
| 2048 | 33.87 ms | 49.20 ms | 45.76 ms | 1.08× |
| 4096 | 77.06 ms | 104.51 ms | 94.70 ms | 1.10× |

is 0.18 blocks per forward over the 3 eligible positions (i.e. blocks 4, 6, or 11 is skipped in roughly 6% of forwards) and LAMBADA accuracy is *above* the full-path retrofit (0.593 vs. 0.576, +1.7pp / +6.1pp over base). The direction (*dynamic skip* $\geq$ *full path*) generalises the Phase-1 zero-degradation finding (Section 6.1; Table 6): at 340M the two were bit-identical, here the skipped retrofit is strictly better. The mechanism is the same—skipping a block whose $\alpha$ has already collapsed onto the immediate predecessor removes a redundant forward, and the calibrated per-block threshold ensures this only fires when the routing pattern itself certifies the block as locally redundant. What changes between Phase 1 and Phase 2 is not the routing rule but the base capability of the backbone: a frozen 2B pretrained model has far more redundant computation to remove than a from-scratch 340M, so the same skip rule moves from "no regression" to "strict improvement". At the aggressive end ($q$=0.5, $M_{\max}$=2) we skip 1.41 blocks on average at the cost of 10pp accuracy—the Pareto cliff steepens once we skip more than one block per token.

**Wall-clock speed.** Table 9 reports forward-pass latency. Retrofit adds ∼35–50% latency over the stock Qwen3-VL-2B (one block-level router + adapter MLP per block, across 14 blocks). Dynamic skip recovers 1.08–1.27× over the retrofit full path, larger at short sequences where per-input variance in $w_{\text{recent}}$ translates into more frequent skip triggers. We explicitly do *not* claim speedup over the unmodified base: our retrofit is a capability-plus-skip trade in which the capability gain absorbs the router cost on most benchmarks. An adapter-rank ablation ($r$=32 vs. $r$=128; Appendix A.7) shows adapter compute is not the bottleneck—router op dominates—so latency is essentially fixed by the routing choice. Scaling to more blocks per router (e.g., $N$=7), or sharing routers across block groups, are natural levers we leave to future work.

**Why 5,000 steps at $\gamma \to 1$, not more.** We characterised the retrofit design space by sweeping 8 variants of the $\gamma \to 1$ recipe over adapter rank ($r \in \{32, 64, 128, 256\}$), training duration (5k/10k/20k), data mixture (50% vs 80% LLaVA), and $\gamma$ ramp-fraction (0.3 vs 0.7). Two findings drove the canonical choice (full table in Appendix A.8):

1. **Training duration at $\gamma$=1 has a strict monotonic effect on multimodal capability.** Holding all other hyperparameters fixed at ($r$=256, 50% VLM, fast ramp), MMBench tracks the number of steps spent at $\gamma$=1: 5k

steps $\Rightarrow 0.710$ ($-1.7$pp), 10k $\Rightarrow 0.587$ ($-14$pp), 20k $\Rightarrow 0.513$ ($-21$pp). LAMBADA is non-monotonic (peaks at 10k, regresses at 20k), so the extra compute past 5k buys no text-domain gains either. The 5k-step budget is the only setting that preserves MMBench within noise; beyond it the router+adapter over-specialise to the training mix and break the vision-language alignment the frozen backbone was originally calibrated for.

2. **Adapter rank and data balance are subordinate levers.** At 10k steps, halving rank ($256 \rightarrow 32$) recovers roughly a third of the MMBench drop but never eliminates it (best 10k setting: $r$=64, MMBench $0.697$, still $-3.0$pp); pushing the mix to $80\%$ VLM *worsens* MMBench at all ranks, indicating the regression is router over-specialisation, not insufficient visual exposure. Slowing the $\gamma$ ramp is catastrophic ($0.517$ MMBench at 10k/$r$=256/slow ramp) because it leaves the backbone fewer steps to re-calibrate to the routed input distribution after $\gamma$ reaches 1.

A separate $\gamma$-free control (identical setup but $\gamma_n \equiv 0$ learnable around its zero-init, $r$=128, 10k steps) achieves LAMBADA $0.576$ / HellaSwag $0.512$ / MMBench $0.720$—the same LAMBADA gain, slightly weaker HellaSwag, slightly stronger MMBench, but the model still runs the original residual path with the adapter acting as a learnable correction. It sits at the opposite structural extreme from the canonical, and we include it in Appendix A.8 as a non-pure ATTNRES baseline demonstrating that the same downstream gain is achievable without pure ATTNRES routing—at the cost of structural fidelity to the paper's claim.

**Per-block skip importance.** Sweeping static single-block removal on the trained 2k retrofit (Appendix A.9) reproduces the importance–ablation disconnect observed in the 340M experiment (Section 4.2): block 1 is catastrophic to remove ($-55\%$ LAMBADA acc) even though it is a "middle" layer; blocks 4, 6, 11 are the safest ($-11$–$14\%$). This structural pattern appears in *both* the from-scratch and retrofitted ATTNRES model, suggesting it is a property of ATTNRES routing rather than of pretraining scale.

**Where Qwen3-VL-2B sits in the current open VLM landscape.** To situate our chosen base, Table 10 collects the published numbers for classic and recent open VLMs at the $0.5$–8B class on the eight benchmarks the community most commonly reports. The table's purpose is to establish that (i) Qwen3-VL-2B is a strong, recent, near-state-of-the-art base at the 2B scale—so a retrofit that preserves its capability is a non-trivial claim; and (ii) the 2B band is already dense (Qwen2-VL-2B $\rightarrow$ InternVL2.5-2B $\rightarrow$ InternVL3-2B $\rightarrow$ Qwen3-VL-2B), so our retrofit's ability to land gains *on top of* the best-in-band base is the load-bearing quality statement. We leave our retrofit row blank in this table: our in-harness evaluation of Qwen3-VL-2B base (Table 7) scores differently from the official VLMEvalKit report (e.g., MMBench $0.727$ at $n$=300 vs. $78.4$ at full-set with their chat-format grading), so our lightweight likelihood-based subset numbers are not directly comparable to the published ones. A full VLMEvalKit run on our retrofit is planned for the camera-ready.

**Limitations of the current results.** (a) Single retrofit target (Qwen3-VL-2B). Generalisation to other VLM scales and to text-only LMs (e.g., Llama-3) is planned follow-up. (b) Training budget: the $\gamma$=1 regime has a narrow sweet spot around 5k steps for this backbone (documented above); whether the same budget transfers linearly to larger backbones, or scales with base parameter count, is open. (c) We report bfloat16 latency on a single H100; aggregate throughput under KV caching at generation time is an important deployment-side measurement we defer to the VLA section. (d) MMStar is a recent benchmark where our $n$=500 draw may undersample the hardest categories.

# 7 VLA Application

## 7.1 Motivation

In Vision-Language-Action (VLA) models, tokens of different modalities serve fundamentally different roles:

- **Vision tokens** encode perceptual features; prior work shows these stabilise in early layers [30].
- **Language tokens** carry task specifications and reasoning.

Table 10: **Open VLM landscape at the 0.5–8B class.** Numbers are taken from each model's own technical report or official model card (primary source). "—" means the primary source did not report that benchmark in a directly comparable form. MMBench column is English v1.1 (test split) where available, else the older MMBench-EN (dev); MMMU is the val split; MathVista is testmini; OCRBench is on a 0–1000 scale. Our retrofit row is left blank: our in-harness eval uses a lightweight likelihood-based rubric on benchmark subsets and is not directly comparable to VLMEvalKit-based published scores; apples-to-apples gains are measured against our own harness and are reported in Table 7.

| Model | #Params | MMB-v1.1 | MMStar | MMMU | AI2D | OCRBench | MathVista | RWQA | ref. |
|---|---|---|---|---|---|---|---|---|---|
| *Small open VLMs ($\leq$3B)* | | | | | | | | | |
| LLaVA-OneVision-0.5B | 0.5B | 43.8 | 36.3 | 31.2 | 54.2 | — | 34.6 | — | [21] |
| Qwen2-VL-2B | 2B | 74.9 | 48.0 | 41.1 | 74.7 | 809 | 43.0 | 62.9 | [34] |
| InternVL2.5-2B | 2B | 72.2 | 53.7 | 43.6 | 74.9 | 804 | 51.3 | 60.1 | [6] |
| InternVL3-2B | 2B | 78.6 | 60.7 | 48.6 | 78.7 | 835 | 57.0 | 64.3 | [39] |
| Qwen2.5-VL-3B | 3B | 77.6 | 55.9 | 53.1 | 81.5 | — | 62.3 | — | [2] |
| **Qwen3-VL-2B (base)** | 2B | 78.4 | 58.3 | 53.4 | 76.9 | 858 | 61.3 | 63.9 | [3] |
| + **AR-Retrofit** (*ours*) | 2B | — | — | — | — | — | — | — | this work |
| *Medium open VLMs (4–8B), for scale context* | | | | | | | | | |
| Phi-3.5-Vision | 4.2B | 81.9 | — | 43.0 | 78.1 | — | 43.9 | — | [1] |
| Gemma-3-4B | 4B | — | — | 48.8 | 74.8 | — | 50.0 | — | [15] |
| Qwen3-VL-4B | 4B | 83.9 | 69.8 | 67.4 | 84.1 | 881 | 73.7 | 70.9 | [3] |
| LLaVA-OneVision-7B | 7B | 80.8 | 61.7 | 48.8 | 81.4 | — | 63.2 | 66.3 | [21] |
| MiniCPM-V-2.6 | 8B | 78.0 | 57.5 | 49.8 | 82.1 | 852 | 60.8 | 65.0 | [35] |
| InternVL3-8B | 8B | 81.7 | 68.2 | 62.7 | 85.2 | 880 | 71.6 | 70.8 | [39] |
| Qwen3-VL-8B | 8B | 84.5 | 70.9 | 69.6 | 85.7 | 896 | 77.2 | 71.5 | [3] |

- **Action tokens** require compositional motor planning and typically benefit from deeper processing.

There is no principled reason for all three modalities to use the same effective depth. Yet in practice all existing open VLA architectures [5, 26, 19] process every token through every layer. ATTNRES's per-position routing offers a mechanism to tailor effective depth per modality without any token-level gating machinery.

## 7.2 Planned experiments (work in progress)

The VLM retrofit of Section 6.2 is already on Qwen3-VL-2B with vision tower frozen, so the first half of the VLA pipeline is in place; what remains is attaching an action head, fine-tuning on trajectory data, and measuring modality-conditioned skip. The numbers below are planned; we report results on Qwen3-VL-2B alone in this draft and will add VLA results and additional retrofit targets in the camera-ready revision.

**Target VLA backbone.** Qwen3-VL-2B with the vision tower frozen (unchanged from Section 6.2). We will attach a flow-matching action head [4] for continuous control and fine-tune on LIBERO-90 [23] and a subset of OpenX-Embodiment [8].

**Analysis: modality-specific routing.** For each held-out LIBERO trajectory we extract the per-token $\alpha$ distribution and average by modality. We expect (hypothesis H1) vision-token $\alpha$ to concentrate on early blocks, (H2) action-token $\alpha$ to spread to late blocks, and (H3) language-token $\alpha$ to sit between. Visualisations will mirror Figure 1(a) per-modality.

**Modality-aware skip.** At inference, we allow different skip policies per modality:

$$\text{skip block } n \text{ for modality } m \iff w_{\text{recent}}^{(m)}(n) > \tau_n^{(m)} \;\text{ and }\; n \in \mathcal{P}^{(m)}, \tag{8}$$

with per-modality $(\mathcal{P}^{(m)}, \tau_n^{(m)}, M_{\max}^{(m)})$. A block is skipped for a token only if the rule fires under that token's modality, so aggressive settings on vision tokens do not starve action tokens. We compare against a uniform skip baseline at matched effective FLOPs.

Table 11: **Open VLA landscape on standard benchmarks.** LIBERO entries are success rates (%) per task suite; SimplerEnv entries are aggregate success rates (%), split by robot setup where both are reported. "—" indicates not reported in the primary source. Following common practice, $\pi_0$ and Octo-Base LIBERO numbers are taken from OpenVLA-OFT [20], which re-ran all three policies under a uniform LIBERO protocol; RT-2-X SimplerEnv numbers are from the SimplerEnv paper [22]. Our target (bottom row) is the retrofit of Section 6.2 with an attached flow-matching action head; trajectory fine-tuning is WIP.

| | | LIBERO (%) | | | | SimplerEnv (%) | | |
|---|---|---|---|---|---|---|---|---|
| Model | #Params | Spatial | Object | Goal | Long-10 | Bridge | Google | ref. |
| Octo-Base | 93M | 78.9 | 85.7 | 84.6 | 51.1 | — | — | [26, 20] |
| RT-2-X | 55B | — | — | — | — | 1.1 | 6.3 | [5, 22] |
| OpenVLA | 7B | 84.7 | 88.4 | 79.2 | 53.7 | — | — | [19, 20] |
| TraceVLA | 7B | 84.6 | 85.2 | 75.1 | 54.1 | — | ∼47.7 | [38] |
| SpatialVLA | 4B | 88.2 | 89.9 | 78.6 | 55.5 | 42.7 | 70.7 | [29] |
| $\pi_0$ | 3B | **96.8** | **98.8** | **95.8** | **85.2** | — | — | [4, 20] |
| **Qwen3-VL-2B + AR-Retrofit + action head** (*ours, planned*) | 2B | — | — | — | — | — | — | this work |

**Target metrics.** LIBERO-90 success rate, action L1 error, inference FLOPs, wall-clock end-to-end latency. The main claim we aim to support: *at matched success rate, modality-aware skip saves* $\geq 15\%$ *more FLOPs than uniform skip.*

**Context: recent open VLAs on standard benchmarks.** To calibrate what "success rate" means at the scale we target, Table 11 collects the published numbers for classic and recent open VLAs on the two evaluation suites the community converges on: **LIBERO** (four task suites—Spatial, Object, Goal, Long-horizon—each as success rate over simulated manipulation rollouts) and **SimplerEnv** (Bridge/WidowX and Google Robot setups, aggregate success rate). These are the benchmarks we plan to evaluate our retrofit on once the action-head training completes. Our row is left blank: no trajectory fine-tune has been run yet.

[TODO: Insert Phase-3 VLA results (Weeks 10–12 of the project timeline).]

# 8 Related Work

**Adaptive computation.** ACT [16] introduced learned halting for recurrent networks. Universal Transformers [9] combine ACT with weight-shared transformer blocks but struggle to differentiate the role of each application of the same block. CALM [32] adds auxiliary classifiers per layer for early exit. Mixture-of-Depths [31] routes individual tokens to different layer subsets with capacity losses. Our work differs by using the AttnRes mechanism itself as the routing signal, without any auxiliary component—and, in the retrofit case, by installing that signal into an existing pretrained transformer.

**Layer pruning and skipping.** Gromov et al. [17] showed that many transformer layers can be removed post-training with modest quality loss. ShortGPT [25] introduces block-influence metrics. LayerSkip [12] combines early exit with self-speculative decoding. These approaches either do static removal (no input dependence) or require invasive training changes; our retrofit preserves the full network and adds only a light fine-tune.

**Residual-connection innovations.** DenseNet [18] connects each layer to all previous via concatenation. Highway networks [33] use gated residuals. AttnRes [13] generalises these with an attention-based weighting, which we adopt. We are the first to exploit AttnRes weights as computation-routing signals and the first to show that the mechanism can be retrofitted into pretrained models.

**Vision-Language-Action models.** RT-2 [5] and Octo [26] established the VLA paradigm. OpenVLA [19] demonstrated generalist manipulation. Pi0 [4] integrated flow-matching action heads. Efficient-VLA work so far has focused

on action chunking [37] and post-hoc quantisation; we contribute the first modality-aware adaptive-depth mechanism for VLAs.

# 9 Discussion

**Summary.** We show that AttnRes's two-phase execution already contains the routing signal adaptive computation needs; that this signal can be calibrated into a dynamic ReSkip that preserves quality while saving $1.19\times$ wall-clock at 340M; and that AttnRes can be *retrofitted* into pretrained transformers through a light, identity-at-init fine-tune. On Qwen3-VL-2B the retrofit (7M trainable params, 2000 steps, backbone frozen) improves LAMBADA by $+4.4$pp / $-18\%$ ppl, matches MMBench and MMStar exactly, and inherits ReSkip's phase-1 dynamic skip at an additional $1.1$–$1.27\times$ wall-clock speedup.

**Limitations.** Current retrofit results are from a single target (Qwen3-VL-2B); extension to other VLM scales and to text-only LMs (e.g., Llama-3) is planned. The VLA evaluation (Section 7) is work in progress: the VLM backbone retrofit is complete, but attaching an action head and measuring modality-conditioned skip on LIBERO/OpenX-Embodiment has not yet been run. Our training budget was capped at $2{,}000$ steps; we observed full-path quality saturate early but did not explore whether longer training further improves the skip-path Pareto. The retrofit introduces a $\sim$35–50$\%$ inference overhead from the block-level router+adapter additions; dynamic skip partially offsets it but does not currently yield net speedup over the unmodified base model—closing that gap (smaller or shared routers, block-granularity sweeps, fused kernels) is near-term follow-up. ReSkip's speedup is most visible at long sequences; for short benchmark inputs, probe overhead is a larger fraction of per-block cost and the speedup shrinks correspondingly. The method inherits AttnRes's block-level granularity; per-token routing remains future work.

**Future work: weight-shared AttnRes (ReLoop).** A natural extension is to share block weights across depth and use position-indexed AttnRes pseudo-queries to differentiate each application of the shared block. Unlike Universal Transformers, the pseudo-queries provide an explicit mechanism for "first pass" vs. "third pass" differentiation. Early validation on a 74M from-scratch model showed the training dynamics differ substantially from standard AttnRes, and achieving competitive quality at Universal-Transformer parameter counts remains an open problem. Combined with our retrofit framework, ReLoop offers a path to parameter-efficient adaptive-depth pretrained models; we leave this investigation to future work.

**Broader impact.** Retrofit reduces the compute cost of deploying adaptive-depth models: rather than re-running pretraining for every model one wants adaptive inference on, a single light fine-tune produces a skip-capable variant. This lowers the barrier to deploying efficient inference on existing models across the research and deployment ecosystem.

# References


[1] Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Harkirat Behl, et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024.

[2] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025.

[3] Shuai Bai et al. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025.

[4] Kevin Black, Noah Brown, Danny Driess, et al. $\pi_0$: A vision-language-action flow model for general robot control. *arXiv:2410.24164*, 2024.

[5] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[6] Zhe Chen, Weiyun Wang, Yue Cao, Yangzhou Liu, Zhangwei Gao, Erfei Cui, Jinguo Zhu, Shenglong Ye, Hao Tian, Zhaoyang Liu, et al. Expanding performance boundaries of open-source multimodal models with model, data, and test-time scaling. *arXiv preprint arXiv:2412.05271*, 2024.

[7] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? Try ARC, the ai2 reasoning challenge. *arXiv:1803.05457*, 2018.

[8] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic learning datasets and RT-X models. `https://robotics-transformer-x.github.io/`, 2024.

[9] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *ICLR*, 2019.

[10] Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Zhi Zheng, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. Enhancing chat language models by scaling high-quality instructional conversations. *arXiv preprint arXiv:2305.14233*, 2023.

[11] Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. Depth-adaptive transformer. 2020.

[12] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acber, Saurabh Akin, Ashish Bhatnagar, et al. Layerskip: Enabling early exit inference and self-speculative decoding. *arXiv preprint arXiv:2404.16710*, 2024.

[13] Daniel Y. Fu et al. Attention residual. *arXiv preprint*, 2025. Original AttnRes paper.

[14] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, et al. A framework for few-shot language model evaluation. `https://github.com/EleutherAI/lm-evaluation-harness`, 2024.

[15] Gemma Team. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.

[16] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

[17] Andrey Gromov, Kushal Tirumala, Hassan Shapourian, Paolo Glorioso, and Daniel A. Roberts. The unreasonable ineffectiveness of the deeper layers. *arXiv preprint arXiv:2403.17887*, 2024.

[18] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. In *CVPR*, 2017.

[19] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Maja Vosshall, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[20] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.

[21] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Yanwei Li, Ziwei Liu, and Chunyuan Li. LLaVA-OneVision: Easy visual task transfer. *arXiv preprint arXiv:2408.03326*, 2024.

[22] Xuanlin Li, Kyle Hsu, Jiayuan Gu, Karl Pertsch, Oier Mees, Homer Rich Walke, Chuyuan Fu, Ishikaa Lunawat, Isabel Sieh, Sean Kirmani, Sergey Levine, Jiajun Wu, Chelsea Finn, Hao Su, Quan Vuong, and Ted Xiao. Evaluating real-world robot manipulation policies in simulation. *arXiv preprint arXiv:2405.05941*, 2024.

[23] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *NeurIPS Datasets and Benchmarks*, 2023.

[24] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *arXiv preprint arXiv:2304.08485*, 2023.

[25] Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. Shortgpt: Layers in large language models are more redundant than you expect. *arXiv preprint arXiv:2403.03853*, 2024.

[26] Octo Model Team. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.

[27] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *ACL*, 2016.

[28] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. *NeurIPS Datasets and Benchmarks*, 2024.

[29] Delin Qu, Haoming Song, Qizhi Chen, Yuanqi Yao, Xinyi Ye, Yan Ding, Zhigang Wang, Jiayuan Gu, Bin Zhao, Dong Wang, and Xuelong Li. SpatialVLA: Exploring spatial representations for visual-language-action model. *arXiv preprint arXiv:2501.15830*, 2025.

[30] Maithra Raghu, Thomas Unterthiner, Simon Kornblith, Chiyuan Zhang, and Alexey Dosovitskiy. Do vision transformers see like convolutional neural networks? *NeurIPS*, 2021.

[31] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024.

[32] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. In *NeurIPS*, 2022.

[33] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks. In *ICML Deep Learning Workshop*, 2015.

[34] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024.

[35] Yuan Yao, Tianyu Yu, Ao Zhang, Chongyi Wang, Junbo Cui, Hongji Zhu, Tianchi Cai, Haoyu Li, Weilin Zhao, Zhihui He, et al. MiniCPM-V: A GPT-4V level multimodal LLM on your phone. *arXiv preprint arXiv:2408.01800*, 2024.

[36] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *ACL*, 2019.

[37] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *RSS*, 2023.

[38] Ruijie Zheng, Yongyuan Liang, Shuaiyi Huang, Jianfeng Gao, Hal Daumé, Andrey Kolobov, Furong Huang, and Jianwei Yang. TraceVLA: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. *arXiv preprint arXiv:2412.10345*, 2024.

[39] Jinguo Zhu, Weiyun Wang, Zhe Chen, et al. InternVL3: Exploring advanced training and test-time recipes for open-source multimodal models. *arXiv preprint arXiv:2504.10479*, 2025.

# A Implementation details

## A.1 Online softmax merge with skip

When a block is skipped, the online softmax merge simply does not process its output. The running state $(\mathbf{s}, m, e)$ is unchanged. The output is mathematically equivalent to an AttnRes model trained without that block contributing—modulo the calibration of downstream $\mathbf{w}_l$.

## A.2 Pseudo-query initialisation (Section 4 from-scratch)

For from-scratch training we use $\mathbf{w}_l \sim \mathcal{N}(0, 0.02)$, yielding near-uniform $\alpha$ at initialisation that specialises over the first few thousand steps.

## A.3 Retrofit hyperparameters (Qwen3-VL-2B)

AdamW with $\beta$=$(0.9, 0.95)$, weight decay 0, gradient clip 1.0. Learning rate $10^{-3}$ for retrofit params (routers, adapters, $\gamma$), cosine schedule with 100-step warmup. Sequence length 1536. bfloat16. Loss weights $\lambda_{\text{kl}}$=1, $\lambda_{\text{ent}}$=0.02, KD temperature 1. Skip-branch sampling: one eligible block chosen uniformly at random per step. Adapter bottleneck rank $r$=128. Position-bias on router keys enabled. Router temperature fixed at 1.

## A.4 Block granularity (Qwen3-VL-2B)

For Qwen3-VL-2B's 28 decoder layers we use $N$=14 blocks of 2 layers. Each block receives one router and one residual adapter; the adapter's bottleneck rank defaults to 128.

## A.5 Identity-at-init details

With $\gamma_n$=0 and the adapter up-projection initialised at $\mathcal{N}(0, 0.02^2)$, the retrofit forward is arithmetically identical to the pretrained model (both produce $x_n = h_{n-1}$, which is passed into the unmodified block). We verified end-to-end: at $n$=500 on LAMBADA the retrofit and the base agree to within bfloat16 evaluation noise (acc 0.530 vs. 0.532; ppl 5.572 vs. 5.547); on MMMU and MMStar they are bit-equal at the answer-choice level.

## A.6 Retrofit ablations

Earlier pilots tried (i) observer-only (no forward change; $\alpha$ trained by distillation), (ii) interpolation at block-input with $\beta$=$\sigma$(logit) driven toward 1, and (iii) pure AttnRes with informed init $\mathbf{w}_n \leftarrow c\,\bar{\mathbf{k}}_{n-1}$ + temperature annealing. (i) produced an $\alpha$ head whose skip decision did not feed back through the forward and collapsed LAMBADA to 0.12 accuracy; (ii) pushed the frozen backbone off-distribution once $\beta$ grew and required pretraining data to recover (defeating the "light fine-tune" premise); (iii) was highly sensitive to the consecutive-layer key-similarity of the pretrained backbone, and did not converge within our training budget. The $\gamma$-gated residual injection of Section 5.2 is the converged recipe.

## A.7 Adapter-rank ablation

Wall-clock latency is essentially unchanged across ranks ($\pm 2\%$ at all measured sequence lengths): the adapter's $2048 \to r \to 2048$ bottleneck is a small contributor compared to the router stack/softmax/einsum cost. Rank therefore affects quality but not speed. The canonical $r$=256 was selected from the $\gamma \to 1$ ablation of Appendix A.8; at equal step budgets lower ranks leave more of the MMBench drop on the table without compensating latency savings.

## A.8 $\gamma \to 1$ retrofit ablation

We swept 8 variants of the canonical $\gamma \to 1$ recipe along four axes: adapter rank $r \in \{32, 64, 128, 256\}$, training steps $\in \{5\text{k}, 10\text{k}, 20\text{k}\}$, LLaVA-Instruct fraction of the mix $\in \{50\%, 80\%\}$, and $\gamma$ ramp-fraction $\in \{0.3, 0.7\}$. All other hyperparameters (loss, optimizer, schedule family, seed) are held constant. A separate $\gamma$-free control is run for the same number of steps with $\gamma_n \equiv 0$ trainable around its zero-init (Appendix Table 12).

Table 12: **Retrofit ablation** on Qwen3-VL-2B. $\gamma$-free: $\gamma_n$ trainable around 0, adapter-dominated (opposite structural extreme to the canonical). All others: $\gamma$-curriculum $0 \to 1$ over the first $30\%$ of steps. "n=300 MMBench noise floor $\pm 1$ question" corresponds to $\pm 0.33$pp.

| Name | $\gamma$ sched | rank | steps | VLM% | ramp | LAMBADA | HellaSwag | MMBench |
|---|---|---|---|---|---|---|---|---|
| Base Qwen3-VL-2B | — | — | — | — | — | 0.532 | 0.506 | 0.727 |
| $\gamma$-free (A) | $\gamma \approx 0$ trainable | 128 | 10k | 50 | — | 0.576 | 0.512 | 0.720 |
| **Canonical** | $0 \to 1$ fast | **256** | **5**k | **50** | **0.3** | **0.576** | **0.522** | **0.710** |
| + low rank | $0 \to 1$ fast | 64 | 10k | 50 | 0.3 | 0.570 | 0.530 | 0.697 |
| + very low rank | $0 \to 1$ fast | 32 | 10k | 50 | 0.3 | 0.568 | 0.526 | 0.683 |
| + low rank, VLM-heavy | $0 \to 1$ fast | 64 | 10k | 80 | 0.3 | 0.560 | 0.524 | 0.660 |
| + mid rank, VLM-heavy | $0 \to 1$ fast | 128 | 10k | 80 | 0.3 | 0.564 | 0.520 | 0.653 |
| + VLM-heavy | $0 \to 1$ fast | 256 | 10k | 80 | 0.3 | 0.560 | 0.520 | 0.617 |
| + longer, 10k | $0 \to 1$ fast | 256 | 10k | 50 | 0.3 | 0.586 | — | 0.587 |
| + slow ramp | $0 \to 1$ slow | 256 | 10k | 50 | 0.7 | 0.568 | 0.520 | 0.517 |
| + longer, 20k | $0 \to 1$ fast | 256 | 20k | 50 | 0.3 | 0.568 | 0.520 | 0.513 |

**Over-training signature at $\gamma$=1.** Holding ($r$=256, $50\%$ VLM, fast ramp) fixed and varying only the total step count, MMBench is strictly monotonic in steps: 5k $\to$ 0.710 (preserved), 10k $\to$ 0.587 ($-$14pp), 20k $\to$ 0.513 ($-$21pp). LAMBADA peaks at 10k (0.586) and regresses at 20k (0.568), so longer training buys no text-domain gain either. The $\gamma$=1 regime has a narrow compute sweet spot; past it the router+adapter over-specialise to the training distribution and the frozen backbone, unable to adapt further, loses its multimodal calibration.

**Rank and data are subordinate.** Halving rank at 10k recovers some MMBench ($r$=32: 0.683, $r$=64: 0.697), but never reaches the 5k-$r$=256 level (0.710). Pushing the mix to $80\%$ LLaVA *worsens* MMBench at every rank, indicating the regression is router over-adaptation to *any* narrow training distribution, not insufficient visual exposure.

**$\gamma$-free as the opposite extreme.** The $\gamma$-free control (row 2) achieves the same LAMBADA gain ($+4.4$pp) as the canonical with slightly weaker HellaSwag ($+0.6$pp vs. $+1.6$pp) and slightly stronger MMBench (0.720 vs. 0.710). It reaches these numbers by treating the adapter as a learnable correction *on top of* the original residual path—the model still computes $x_n \approx h_{n-1} + \text{adapter}(\cdot)$, never structurally AttnRes. Because the paper's claim is that pretrained transformers can be rewired *into* the AttnRes form, we take the $\gamma \to 1$ canonical as our headline and report this row as a non-pure-AttnRes control showing the same gain is achievable with the standard residual path left in place.

## A.9 Per-block skip importance

Running the 2k Qwen3-VL-2B retrofit with a single block statically removed ($h_n \leftarrow x_n$) and measuring LAMBADA at $n$=300: block 1 is catastrophic ($-55\%$ acc), block 10 severe ($-46\%$), blocks 4, 6, 11 are safest ($-11$ to $-14\%$). We use this sweep to pick $\mathcal{P}=\{4, 6, 11\}$ for the dynamic-skip eligibility set.

## A.10 Calibration set (dynamic skip)

Calibration uses 32 held-out LAMBADA prefixes (truncated to 512 tokens each). Per-block $w_{\text{recent}}(n)$ samples are collected with a single retrofit forward and the empirical quantile at $q$ is used as $\tau_n$. We verified that varying the calibration draw shifts $\tau_n$ by $< 0.01$ in absolute terms at the chosen $q$=0.95.